Business Intelligence Systems

INFO 4555 5556

Fall 2025

Data Profiling



Cornell University

Data Profiling

"Data profiling refers to the analysis of information for use in a data warehouse in order to clarify the structure, content, relationships, and derivation rules of the data." – Ralph Kimball

Analysis of data being considered as a source to the data warehouse.

What is evaluated?

- Data Structure (ex. Data Type, Data Length, constraints)
- Uniqueness
- Occurrence of null values
- Distribution of values (high, low, mean, median, counts of distinct values)
- Relationships between attributes (possible imbedded value dependencies)
- Check for illegal values, misspellings, missing values, duplicates
- Evaluation of join paths between tables



Benefits of Data Profiling:

- Improve understanding of organization's data
- Improve quality of data (by identifying data quality issues)
- Determine whether source system data will support BI requirements
- Results in metadata useful to DW
- Speeds the data modeling and ETL process

When poll is active, respond at **pollev.com/info4555**

Text **INFO4555** to **22333** once to join

All of the following are true about data profiling except:

A: Help identify whether source system data will support BI requirements

B: Used to identify data quality issues

C: Cannot be performed with SQL

D: Generates useful metadata about source data

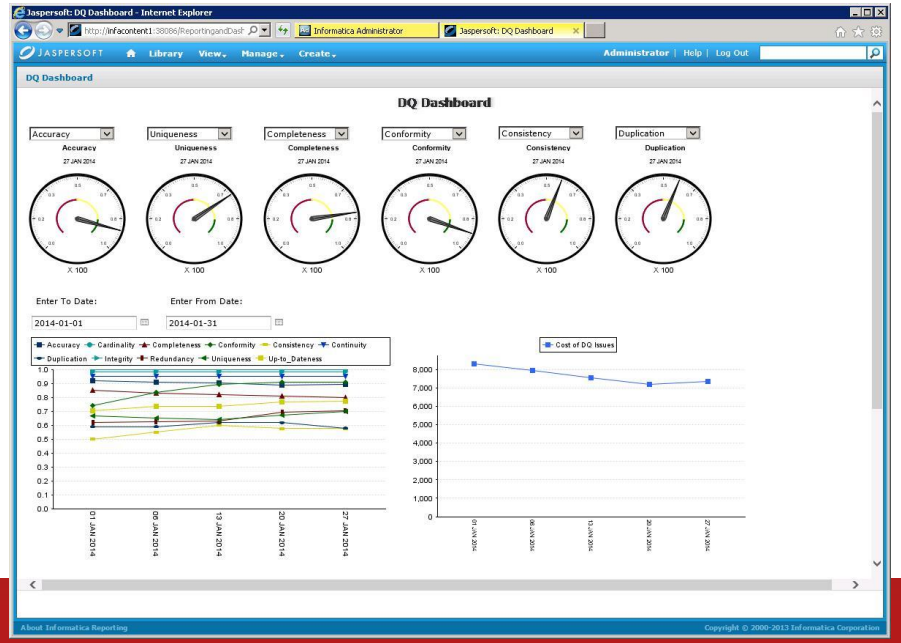
E: Generally speeds up data modeling and ETL development

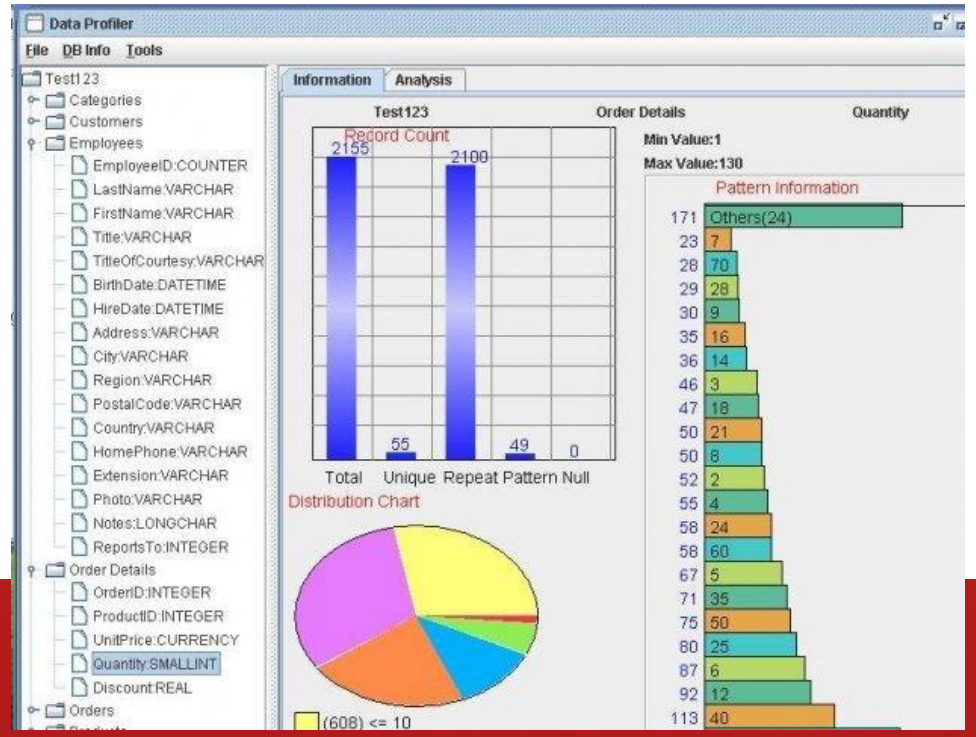
Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Data Profiling Tools

There are many Data Profiling tools on the market to automate many aspects of the data profiling process.

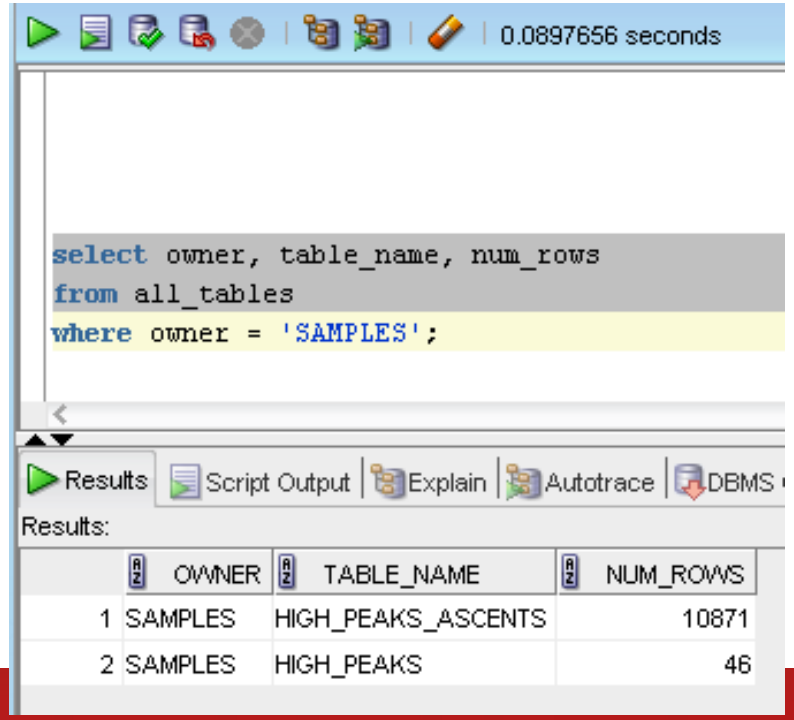
In this class we will be using SQL for our data profiling.





What is the scope of tables?

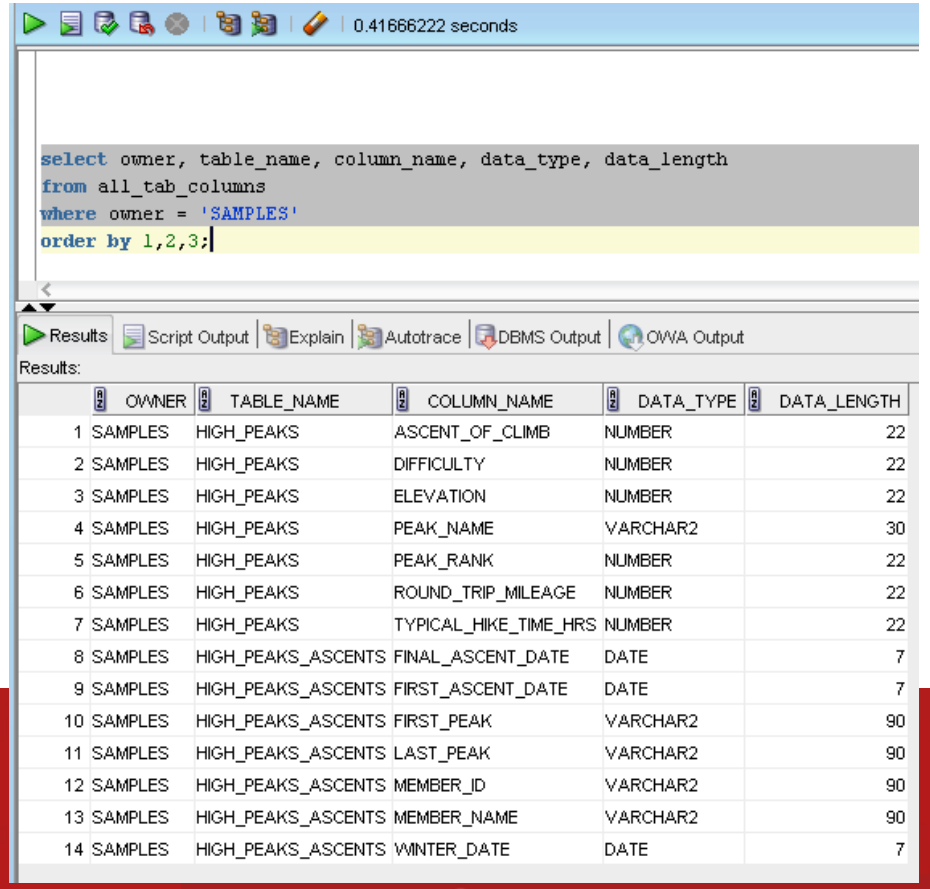
The following query will return all of the tables, and their record counts, for a particular schema.



For schemas with many tables, may also want to include filter on num_row such as: Num_rows > 0;

Column structure information

This query will return the table and columns names, along with data type information, for a particular schema.



	OWNER	TABLE_NAME	COLUMN_NAME	DATA_TYPE	DATA_LENGTH
1	SAMPLES	HIGH_PEAKS	ASCENT_OF_CLIMB	NUMBER	22
2	SAMPLES	HIGH_PEAKS	DIFFICULTY	NUMBER	22
3	SAMPLES	HIGH_PEAKS	ELEVATION	NUMBER	22
4	SAMPLES	HIGH_PEAKS	PEAK_NAME	VARCHAR2	30
5	SAMPLES	HIGH_PEAKS	PEAK_RANK	NUMBER	22
6	SAMPLES	HIGH_PEAKS	ROUND_TRIP_MILEAGE	NUMBER	22
7	SAMPLES	HIGH_PEAKS	TYPICAL_HIKE_TIME_HRS	NUMBER	22
8	SAMPLES	HIGH_PEAKS_ASCENTS	FINAL_ASCENT_DATE	DATE	7
9	SAMPLES	HIGH_PEAKS_ASCENTS	FIRST_ASCENT_DATE	DATE	7
10	SAMPLES	HIGH_PEAKS_ASCENTS	FIRST_PEAK	VARCHAR2	90
11	SAMPLES	HIGH_PEAKS_ASCENTS	LAST_PEAK	VARCHAR2	90
12	SAMPLES	HIGH_PEAKS_ASCENTS	MEMBER_ID	VARCHAR2	90
13	SAMPLES	HIGH_PEAKS_ASCENTS	MEMBER_NAME	VARCHAR2	90
14	SAMPLES	HIGH_PEAKS_ASCENTS	WINTER_DATE	DATE	7

Combining the table and columns views

You may simply combine the table and column data by joining the all_tables and all_tab_columns catalog views. Note that the join has two attributes: owner and table_name.

0.68198299 seconds

```
select t.owner,t.table_name, tc.column_name, tc.data_type, tc.data_length, t.num_rows
from all_tables t, all_tab_columns tc
where T.TABLE_NAME = tc.table_name
and t.owner = tc.owner
and T.OWNER = 'SAMPLES'
order by 2,3;
```

Results | Script Output | Explain | Autotrace | DBMS Output | OWA Output

Results:

	OWNER	TABLE_NAME	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	NUM_ROWS
1	SAMPLES	HIGH_PEAKS	ASCENT_OF_CLIMB	NUMBER	22	46
2	SAMPLES	HIGH_PEAKS	DIFFICULTY	NUMBER	22	46
3	SAMPLES	HIGH_PEAKS	ELEVATION	NUMBER	22	46
4	SAMPLES	HIGH_PEAKS	PEAK_NAME	VARCHAR2	30	46
5	SAMPLES	HIGH_PEAKS	PEAK_RANK	NUMBER	22	46
6	SAMPLES	HIGH_PEAKS	ROUND_TRIP_MILEAGE	NUMBER	22	46
7	SAMPLES	HIGH_PEAKS	TYPICAL_HIKE_TIME_HRS	NUMBER	22	46
8	SAMPLES	HIGH_PEAKS_ASCENTS	FINAL_ASCENT_DATE	DATE	7	10871
9	SAMPLES	HIGH_PEAKS_ASCENTS	FIRST_ASCENT_DATE	DATE	7	10871
10	SAMPLES	HIGH_PEAKS_ASCENTS	FIRST_PEAK	VARCHAR2	90	10871
11	SAMPLES	HIGH_PEAKS_ASCENTS	LAST_PEAK	VARCHAR2	90	10871
12	SAMPLES	HIGH_PEAKS_ASCENTS	MEMBER_ID	VARCHAR2	90	10871
13	SAMPLES	HIGH_PEAKS_ASCENTS	MEMBER_NAME	VARCHAR2	90	10871
14	SAMPLES	HIGH_PEAKS_ASCENTS	WINTER_DATE	DATE	7	10871

Single attribute / column profiling

The following SQL returns key profile data for a single column:

```
select 'FIRST_ASCENT_DATE' column_name, count(*)
record_count, count(distinct first_ascent_date)
distinct_values_count,
round(count(distinct first_ascent_date) / count(*), 2)*100
percent_unique_values ,
 sum(decode(first_ascent_date, null,1,0)) null_value_count,
round( sum(decode(first_ascent_date,
null,1,0))/count(*),2)*100 percent_null_values
from samples.high_peaks_ascents;
```

COLUMN_NAME	RECORD_COUNT	DISTINCT_VALUES_COUNT	PERCENT_UNIQUE_VALUES	NULL_VALUE_COUNT	PERCENT_NULL_VALUES
FIRST_ASCENT_DATE	10871	5070	47	35	0

COLUMN_NAME	RECORD_COUNT	DISTINCT_VALUES_COUNT	PERCENT_UNIQUE_VALUES	NULL_VALUE_COUNT	PERCENT_NULL_VALUES
FIRST_PEAK	10871	47	0	0	0

Basic statistics for numeric columns

This SQL returns basics stats for a numeric column:

```
select 'ASCENT_OF_CLIMB' column_name,
min(ascent_of_climb) minimum_value,
max(ascent_of_climb) maximum_value,
   round(avg(ascent_of_climb)) mean_value,
median(ascent_of_climb) median_value
from samples.high_peaks;
```

COLUMN_NAME	MINIMUM_VALUE	MAXIMUM_VALUE	MEAN_VALUE	MEDIAN_VALUE
ASCENT_OF_CLIMB	1844	4500	3096	3050

COLUMN_NAME	MINIMUM_VALUE	MAXIMUM_VALUE	MEAN_VALUE	MEDIAN_VALUE
ELEVATION	3820	5344	4405	4380.5



Distribution of values in a column

This SQL will return the record counts for each unique value in a column:

```
select last_peak, count(*)
from SAMPLES.HIGH_PEAKS_ASCENTS
group by last_peak
order by 2 desc;
```

LAST_PEAK	COUNT(*)
Whiteface	1251
Allen	1005
Haystack	768
Colden	480
Skylight	437
Seymour	420
Marcy	404
Rocky Peak	376
Couchsachraga	342
Emmons	341
Sawteeth	331
Marshall	323
Iroquois Peak	316
Saddleback	293
Dix	275
Big Slide	275
Redfield	268
Esther	259
Santanoni	227
Panther	201
Cliff	189
Gray	176
Basin	175
Cascade	149

Column Dependencies / Validating Business Rule

An example of business logic found in a table would be the winter_date column and its relationship with the member_id column in the high_peaks_ascents table. When a climber has climbed all 46 peaks, they are assigned a member_id which is a number designating the order in which they became a member relative to other members. However, the member_id attribute is not numeric in this table, because a member may have a "W" suffix on their member_id, if they later climb all 46 peaks during the winter months. Also, if a member does climb all 46 peaks during the winter months, the date at which they complete their final winter peak is recorded in the winter_date column. For most members the winter_date column is null, unless they are a "Winter 46er".

MEMBER_ID	MEMBER_NAME	FIRST_PEAK	FIRST_ASCENT_DATE	LAST_PEAK	FINAL_ASCENT_DATE	WINTER_DATE
776	Donald Federman	TableTop	30-AUG-69	Gray	28-MAY-72	(null)
777	Brian Aveny	Lower Wolf Jaw	03-SEP-66	Seymour	29-MAY-72	(null)
778W	Roger L. Harris	Big Slide	01-SEP-69	Emmons	05-JUN-72	21-FEB-76
779	Craig P. Palmer	Giant	30-MAY-71	Whiteface	10-JUN-72	(null)
780	James Underwood	Dix	01-JUL-67	Esther	17-JUN-72	(null)
781	Sean Cridland	Algonquin	23-JUN-71	Emmons	07-JUL-72	(null)
782W	Richard Mallison	Grace Peak	27-SEP-70	Esther	11-JUL-72	11-MAR-88
783	Howard H. Hansen	Lower Wolf Jaw	07-JUN-69	Nye	12-JUL-72	(null)
784	David G. Butterworth	Big Slide	17-JUL-67	Colden	13-JUL-72	(null)

Member_id's with a "W"suffix have a valid date in the winter_date column

Column Dependencies / Validating Business Rule

To verify this relationship between the member_id and winter_date columns, you will need to verify that all member_id's with the "W" suffix, also have a non-null winter_date value. Also, that all member_id's without a "W" suffix have a null value for their winter_date attribute.

```
select *
from SAMPLES.HIGH_PEAKS_ASCENTS
where member_id not like '%W' -- not a winter member_id
and winter_date is not null  --aka has a date
;
```

Results | Script Output | Explain | Autotrace | DBMS Output | OWA Output

Results:

MEMBER_ID	MEMBER_NAME	FIRST_PEAK	FIRST_ASCENT_DATE	LAST_PEAK	FINAL_ASCENT_DATE	WINT

Test 1. Are there any records a member has a winter_date, but don't have a "W" suffix on their member_id. No records returned.

```
select *
from SAMPLES.HIGH_PEAKS_ASCENTS
where member_id like '%W' -- is a winter member_id
and winter_date is null  --does not have a winter_date assigned
;
```

Results | Script Output | Explain | Autotrace | DBMS Output | OWA Output

Results:

	MEMBER_ID	MEMBER_NAME	FIRST_PEAK	FIRST_ASCENT_DATE	LAST_PEAK	FINAL_ASCENT_DATE	WINT
1	5234W	Michael A. Strich	Cascade	11-JUN-94	Allen	01-JAN-15	(null)

Test 2. Are there any records a member does not have a winter_date, but has a "W" suffix on their member_id. One record is not compliant to the business rule.

Test table join paths

Make sure the primary keys are unique and do not contain null values.

Note that if the column has a primary key constraint in the database, the database will enforce uniqueness and not-null for the column.

```
select peak_name, count(*)
from samples.high_peaks
group by peak_name
having count(*) > 1;
```

This query will only return records with duplicate values. In this case no records are returned. So, the column is unique.

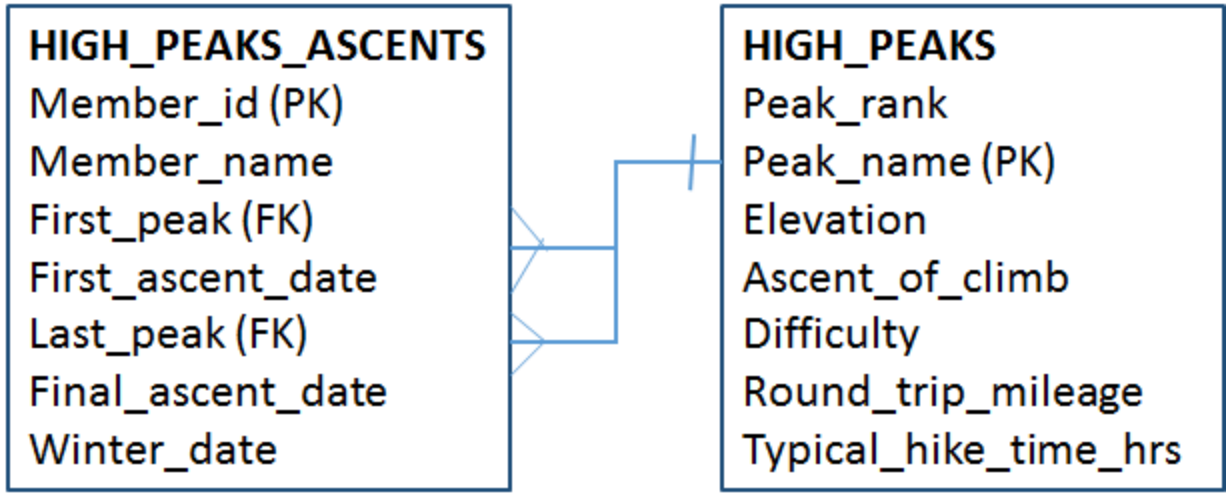
```
Select count(*)
from samples.high_peaks
where peak_name is null;
```

This query counts records with a null value for the specified column. The record count is 0, so the column contains NO null values.

Test table join paths (continued)

Test the foreign keys. All values in the foreign_key column should have a match in the primary_key you intend to join to.

Let's test the join between high_peaks_ascents.first_peak and high_peaks.peak_name.



Test table join paths (continued)

```
select count(*)
from samples.high_peaks_ascents;
```

Results | Script Output | Explain | Autotr

Results:

	COUNT(*)
1	10871

First lets get a count of records for the driving table. In this case "high_peaks_ascents".

```
select count(*)
from samples.high_peaks_ascents hps, samples.high_peaks hp
where hps.first_peak = hp.peak_name
;
```

Results | Script Output | Explain | Autotrace | DBMS Output | OWA O

Results:

	COUNT(*)
1	10862

Second, we will attempt to join the two tables, with a simple inner join and get a record count. Notice that we lose 9 records when we join the two tables.

Test table join paths (continued)

```
select first_peak
from samples.high_peaks_ascents
minus
select peak_name
from samples.high_peaks
;
```

Results | Script Output | Explain

Results:

	FIRST_PEAK
1	Unknown

First we select the first_peak value from the high_peaks_ascents table, then we select the peak_name from the high_peaks table. We use a MINUS function between the two queries and the output gives us any records that exist in the first query, but not in the second query. In this case we can see that value "Unknown" does not exist in high_peaks.peak_name.

```
select count(*)
from samples.high_peaks_ascents
where first_peak = 'Unknown';
```

Results | Script Output | Explain

Results:

	COUNT(*)
1	9

Next we see how many instances of the value "Unknown" there are in the high_peaks_ascents.first_peak column. There are 9, which matches the number of records "lost" when we join the two tables. We can change the join to an outer join and will not lose records, but the corresponding values for the 9 records that do not match will be null. So, we will need to figure out how to handle these null values in our ETL.

When poll is active, respond at **pollev.com/info4555**

Text **INFO4555** to **22333** once to join

How do you feel about Data Profiling?

A: Great! Looks pretty simple

B: Good. I think I can start applying the concepts and SQL to project.

C: OK. I'll figure it out once I review the slides and try the SQL.

D: Bad. Totally lost me.

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**